Higher-MI Predictor

Population O-Information on the Logits of a Grokking Transformer

A Unified Benchmark of Grokking Predictors in Neural Networks

Predictor 7 of 8

Prepared for Jonathan John

M.Sc. Artificial Intelligence, IU Internationale Hochschule

20 October 2026

Status. Higher-MI is CLOSED, negative. The event fires after the grok epoch in 5 of 5 seeds. Frozen-protocol numbers: lead_frac $= 0$, $\rho = 0.7$, mean_rel_lead $= -0.4254$.

How to Read This Document

This document explains the Higher-MI predictor as it is implemented in this thesis project. It covers what the predictor measures, how the code computes it, and what the results mean.

Prerequisite. You have already studied information theory. So entropy H, mutual information $I(X;Y)$ and conditional mutual information $I(X;Y \mid Z)$ are assumed known. They are stated where needed, but they are not derived. No worked information-theory examples are given.

What is explained in full. Everything that is specific to this experiment is explained in full. This includes the task, the test set, the logits, the three variables F_0, F_1, F_2, the KSG estimator as used in the code, the two bugs, the event rule, the frozen evaluation protocol, the results and their meaning.

Order of chapters. Every word is defined before it is used. For this reason, the whole-pipeline picture comes only in Chapter 9. By that point, every box in the picture has already been defined.

Sources of every number. All numbers come from the project's own saved files. The main sources are `08_Experiments/results/nanda_unified/seed_<N>/higher_mi/higher_mi_signal.json` and `aggregate_higher_mi.json`. The single-example numbers in Chapter 3 were computed fresh from the saved seed-0 checkpoints while preparing this document. They reproduce the saved Ω_{pop} values exactly (Section 3.7).

Contents

1 Where Higher-MI Sits in the Thesis **1**

1.1 The Thesis Question 1

1.2 The Benchmark Line-Up 1

1.3 The Source Paper 1

2 The Experiment: Every Word Defined Before Use **3**

2.1 The Task 3

2.2 Training Set and Test Set 3

2.3 The Model 4

2.4 Classes and Logits 4

2.5 Checkpoints, Seeds and the Grok Epoch 5

3 From 113 Logits to Three Numbers: F_0, F_1, F_2 **6**

3.1 Why We Reduce 113 Logits to Three 6

3.2 Main Relation (A) 6

3.3 Term (i): F_0, the Correct-Class Logit 6

3.4 Term (ii): F_1, the Runner-Up Logit 6

3.5 Term (iii): F_2, the Rest-Mean Logit 7

3.6 The Same Test Example at Three Checkpoints 7

3.7 From One Test Example to the Whole Test Set 8

3.8 The Code That Does This 8

4 O-Information (Brief) **9**

4.1 Definition 9

4.2 Term Extraction on Real Numbers 9

4.3 Averaging Over the Six Orderings 10

4.4 The Hypothesis Being Tested 10

5 The KSG Estimator **11**

5.1 Why an Estimator Is Needed 11

5.2 How It Works, Step by Step 11

5.3 Choice of k 11

5.4 Code Functions 12

5.5 Built-In Sanity Checks 12

6 Population O-Information versus Temporal O-Information **13**

6.1 What “Samples” Means in the Paper 13

6.2 What “Samples” Means in This Project 13

6.3 The Risk: Shared Example Difficulty . . . 13
6.4 The Temporal Control . . . 14
6.5 Transfer Entropy (Exploratory Only) . . . 14

7 The Event Rule: From a Curve to One Epoch 15
7.1 Why an Event Rule Is Needed . . . 15
7.2 The Primary Rule: Global Maximum . . . 15
7.3 Exploratory Rules . . . 15

8 The Frozen Evaluation Protocol 16
8.1 Quantities . . . 16
8.2 The PASS Rule . . . 16
8.3 Why $\rho \geq 0.9$: Calibration . . . 16

9 The Whole Pipeline in One Picture 18
9.1 Where Each Box Lives in the Code . . . 18
9.2 Output Files per Seed . . . 19

10 Two Bugs Found and Fixed 20
10.1 Bug 1: Inclusive versus Strict Neighbour Counting . . . 20
10.2 Bug 2: The Test Split Was Not Re-Seeded . . . 20

11 Results 22
11.1 Raw Curves First . . . 22
11.2 Population versus Temporal Control . . . 24
11.3 Primary Event per Seed . . . 25
11.4 Frozen-Protocol Scoring . . . 25
11.5 Exploratory Signals (Not Scored) . . . 27

12 What the Negative Result Means 28
12.1 The Main Conclusion . . . 28
12.2 What the Raw Curves Tell Us . . . 28
12.3 Comparison with the Paper's Own Claim . . . 28
12.4 What the Result Does Not Mean . . . 28
12.5 Thesis Wording (Ready to Use) . . . 28

13 Assumptions, Limitations and Open Items 29
13.1 Assumptions Made in the Adaptation . . . 29
13.2 Limitations . . . 29
13.3 Open Items . . . 29

14 How to Reproduce 30

A Symbols **31**

Chapter 1: Where Higher-MI Sits in the Thesis

1.1 The Thesis Question

Let us first recall the question this thesis asks.

Grokking is a delayed generalisation. The model first fits the training set perfectly. Test accuracy stays low for a long time. Then, much later, test accuracy suddenly rises.

Many papers propose a quantity that is said to "track" or "detect" grokking. This thesis asks one strict question about each such quantity:

Does the quantity give a warning *before* grokking happens, under one common protocol?

A quantity that only moves at the same time as grokking, or after it, cannot be used as an early warning.

1.2 The Benchmark Line-Up

The predictors are evaluated in a fixed order. Correlation Traps was removed on 30 September 2026, because it targets anti-grokking (collapse after grokking), not the onset of grokking. So there are now 8 predictors.

No.	Predictor	Kind of quantity	Status
1	L2 Norm	weights	CLOSED, negative
2	Dropout	robustness of outputs	CLOSED, negative
3	Spectral	kernel / task alignment	CLOSED, negative
4	AGE	representation geometry	CLOSED, negative
5	HTSR Alpha	weight spectrum	CLOSED, negative
6	Weight-PCA	weight effective rank	CLOSED, negative
7	**Higher-MI**	**information among output logits**	**CLOSED, negative**
8	Commutator Defect	—	next, not started

Table 1.1: Predictor evaluation order and status on 20 October 2026.

Notice one thing in the third column. Predictors 1, 3, 5 and 6 look at the weights. Predictor 4 looks at hidden representations. Higher-MI is different. It looks only at the *output logits* of the model. It asks how three output quantities share information with each other.

1.3 The Source Paper

The predictor comes from:

D. Pomarico, R. Cilli, A. Monaco, *et al.*, "Transfer entropy and O-information to detect grokking in tensor network multi-class classification problems," arXiv:2507.23346 (2025).

Why: We need to know what the paper actually did. Only then can we judge what our adaptation changes.

What: The paper trains Matrix Product State (MPS) classifiers. An MPS is a tensor-network model from quantum many-body physics. It is not a transformer. The paper uses two three-class problems.

One is a reduced Fashion-MNIST (dress, sneaker, bag). The other is hyperspectral satellite land-cover data.

The paper has exactly three classes. So the model gives exactly three output scores at each training sweep. The paper computes the O-information among these three scores over the training sweeps.

So what: Two points from the paper matter for us.

Point 1. The paper reports that O-information is strongly negative at random initialisation. It then jumps positive and peaks at the grokking transition of Fashion-MNIST. Its section heading is "Scores redundancy peak at grokking."

Point 2. The paper's own wording is that the peak *coincides* with grokking. The paper never claims that the peak comes *before* grokking.

So the paper itself makes a "coincides" claim, not a "leads" claim. This thesis tests the stronger "leads" claim. We will come back to this in Chapter 12.

Chapter 2: The Experiment: Every Word Defined Before Use

Before we touch O-information, let us fix every experiment word. Each definition is given once. After that, the word is used freely.

2.1 The Task

Definition 2.1 (Task: modular addition).

Why: Grokking was first seen on small algorithmic tasks. Nanda et al. (2023) made modular addition the standard substrate.

What: The input is a pair of numbers (a,b) with $a,b \in \{0,1,\ldots,112\}$. The output is one number c.

Formula:

$$c = (a+b) \bmod 113.$$

So what: There are exactly 113 possible answers. 113 is a prime, as in Nanda et al.

Example 2.1.

Take $a = 23$ and $b = 45$. Then $23+45 = 68$. Since $68 < 113$, we get $c = 68$.

Definition 2.2 (Input sequence).

Why: A transformer reads a sequence of tokens, not a pair of numbers.

What: Each pair is written as the three-token sequence "a b =". Number tokens have ids 0 to 112. The "=" token has id 113. So the vocabulary size is 114.

So what: The answer is read at the third position, that is, at the "=" token (position index 2).

2.2 Training Set and Test Set

Definition 2.3 (All pairs).

What: Every ordered pair (a,b) is one data point.

Formula:

$$113 \times 113 = 12769 \text{ pairs.}$$

Definition 2.4 (Train/test split).

Why: Grokking is about generalisation. So some pairs must be hidden from training.

What: The 12769 pairs are split randomly. 30% go to training. The rest go to the test set. The split is made by `random_split` in `get_dataloaders()`.

Formula:

$$\begin{aligned} \text{train size} &= \lfloor 0.3 \times 12769 \rfloor \\ &= \lfloor 3830.7 \rfloor \\ &= 3830 \end{aligned}$$

$$\begin{aligned}\text{test size} &= 12769 - 3830 \\ &= 8939\end{aligned}$$

So what: The model trains on 3830 pairs in one full batch. It never sees the other 8939 pairs during training.

Definition 2.5 (Test example).

Why: Higher-MI is computed only on unseen data. So we must be clear what one unseen data point is.

What: Test example matlab: one pair (a,b) that was *not* shown in training. It is one of the 8939 pairs in the test set. It is used to check generalisation, not memorisation.

So what: For seed 0, the pair $(23,45)$ is a test example. This was checked directly on the reconstructed seed-0 split. We will follow this one test example through Chapter 3.

2.3 The Model

Definition 2.6 (Model: Nanda-Unified transformer).

Why: All predictors must be measured on one common model. Otherwise the comparison is not fair.

What: The model is fixed by `06_Code/configs/nanda_unified.yaml`.

Layers: 1.
Attention heads: 4, each of dimension 32.
Model width $d_{\text{model}} = 128$.
MLP width $d_{\text{mlp}} = 512$, ReLU.
No LayerNorm.
Learned position embeddings.
Untied embedding and unembedding.
Initial weights: $\mathcal{N}\left(0,\, 0.8/\sqrt{128}\right)$, standard deviation about 0.0707.
Optimiser: AdamW, learning rate 10^{-3}, weight decay 1.0, $\beta = (0.9, 0.98)$.
Full-batch training, no warm-up, 40000 epochs.

So what: This is a faithful copy of the Nanda et al. (2023) setup. The file `transformer.py` (here `transformer_four_head.py`) is the shared model architecture. Higher-MI did not change it.

2.4 Classes and Logits

Definition 2.7 (Class).

Why: The model must choose one answer. Each possible answer is called a class.

What: There are 113 valid classes: $0, 1, \ldots, 112$. For the test example $(23,45)$, the correct class is $c = 68$. The other 112 classes are wrong.

So what: The output layer actually has 114 outputs, one per vocabulary token. Output 113 belongs to the "=" token. It is never a target label, so the model is never trained to produce it. The Higher-MI code removes it before any calculation (Section 3.8).

Definition 2.8 (Logit).

Why: We need a name for the raw number the model gives to each class.

What: Logit matlab: the official name for the raw score of a class before softmax. For one test example, the model gives one logit per class. We write $\ell[j]$ for the logit of class j.

So what: A higher logit means more belief in that class. A logit is not a probability. It can be any real number, even negative. The predicted class is the class with the largest logit.

2.5 Checkpoints, Seeds and the Grok Epoch

Definition 2.9 (Checkpoint).

Why: Higher-MI is computed after training, from saved models. No retraining is needed.

What: A checkpoint is the saved model state at one epoch of one seed. Checkpoints are saved every 100 epochs, plus the final epoch.

Formula:

$$\text{checkpoint epochs} = \{0, 100, 200, \ldots, 39900\} \cup \{39999\}.$$

That gives $400 + 1 = 401$ checkpoints per seed.

So what: Higher-MI gives one value per checkpoint. So each seed gives a curve with 401 points.

Definition 2.10 (Seed).

Why: One training run can be lucky or unlucky. So the benchmark repeats training 5 times.

What: A seed is the starting state of the random-number generator. Seeds $0, 1, 2, 3, 4$ are used. The seed fixes both the initial weights and the train/test split.

So what: Different seeds grok at very different epochs. This spread is what lets us test whether a predictor's timing follows the grok timing.

Definition 2.11 (Grok epoch).

Why: We need one number for "when grokking happened".

What: The grok epoch G is the first epoch at which test accuracy exceeds 0.9.

So what: The five grok epochs are given below. They are used in every later chapter.

Seed	0	1	2	3	4
Grok epoch G	12987	6921	9510	11019	23747

Table 2.1: Grok epochs of the 5-seed Nanda-Unified run (first epoch with test accuracy > 0.9).

Memorisation is very fast in this run. Training accuracy first reaches 0.99 at about epoch 205 in all five seeds. So the long gap between epoch 205 and the grok epoch is the "memorised but not generalised" phase.

Chapter 3: From 113 Logits to Three Numbers: F_0, F_1, F_2

3.1 Why We Reduce 113 Logits to Three

Why: O-information is a quantity about exactly three variables in its basic form. The source paper had a three-class problem, so it had exactly three scores. Our task has 113 classes. So we need a rule that turns 113 logits into three numbers.

What: The rule keeps the three quantities that matter most for the decision:

the logit of the correct class,
the logit of the strongest wrong class,
the average logit of all the other classes.

So what: This is a project-level adaptation. It is not in the source paper. We record it as an assumption (Chapter 13).

3.2 Main Relation (A)

For one test example with correct class c, the reduction is:

$$\big(\ell[0],\ell[1],\ldots,\ell[112]\big) \;\longmapsto\; \big(F_0, F_1, F_2\big) \tag{A}$$

where

$$\begin{aligned} F_0 &= \ell[c], \\ F_1 &= \max_{j\neq c}\ \ell[j], \\ F_2 &= \frac{1}{111}\sum_{j\neq c,\ j\neq r} \ell[j], \qquad r = \arg\max_{j\neq c} \ell[j]. \end{aligned}$$

Let us take out each term of (A) one by one. We use the test example $(23, 45)$, correct class $c = 68$, seed 0, at checkpoint epoch 12900. This checkpoint is just before the grok epoch 12987.

3.3 Term (i): F_0, the Correct-Class Logit

Why: We want to know how strongly the model believes in the right answer.

What: Given that the true class is $c = 68$, the logit for the correct class will be:

$$F_0 = \ell[68] = 70.292.$$

So what: This is the score of the right answer. On its own it does not tell us whether the answer is predicted. For that, we must compare it with the best wrong class.

3.4 Term (ii): F_1, the Runner-Up Logit

Why: A prediction is correct only if the right class beats every wrong class. So we need the strongest competitor.

What: Given that class 68 is removed, the largest of the remaining 112 logits will be:

$$F_1 = \max_{j \neq 68} \ell[j] = 64.925 \qquad \text{(class 54)}.$$

So what: The margin is

$$\begin{aligned} F_0 - F_1 &= 70.292 - 64.925 \\ &= 5.367. \end{aligned}$$

The margin is positive, so this test example is predicted correctly at epoch 12900.

3.5 Term (iii): F_2, the Rest-Mean Logit

Why: We also want a summary of the "background". This is the level of all the classes that are neither right nor the main competitor.

What: Given that class 68 and the runner-up class 54 are both removed, the average of the remaining 111 logits will be:

$$F_2 = \frac{1}{111} \sum_{j \neq 68,\ j \neq 54} \ell[j] = 29.417.$$

So what: The background sits far below both F_0 and F_1. So at this checkpoint the model has a clear "two-horse race" for this example.

How the code computes it. The code does not mask and then average. It uses the exact identity

$$F_2 = \frac{\sum_{j=0}^{112} \ell[j] - F_0 - F_1}{113 - 2}.$$

A mean taken after masking with $-\infty$ would itself be $-\infty$. The identity avoids that problem.

3.6 The Same Test Example at Three Checkpoints

Let us now follow the same test example $(23, 45) \to 68$ through training.

Epoch	F_0	F_1 (class)	F_2	$F_0 - F_1$	Predicted correctly?
5000	80.854	117.230 (58)	90.024	−36.376	No
12900	70.292	64.925 (54)	29.417	5.367	Yes
20000	137.483	121.685 (84)	66.863	15.798	Yes

Table 3.1: One test example, seed 0. Computed from the saved checkpoints.

What do we see here?

At epoch 5000 the model has memorised the training set. But for this unseen pair it prefers class 58 over the right class 68. The background F_2 is even above F_0.

At epoch 12900 the right class has moved to the top. The background has dropped far below.

At epoch 20000, after grokking, the margin has grown to about 15.8.

The logits in this model are large, of the order of 30 to 150. This is a property of this trained model. It does not affect O-information, because mutual information does not change under rescaling of a variable.

3.7 From One Test Example to the Whole Test Set

Why: O-information is a property of random variables, not of one point. So we need many observations of (F_0, F_1, F_2).

What: At one fixed checkpoint, we compute the triplet for every test example. For seed 0 this gives 8939 triplets. We then treat F_0, F_1, F_2 as three random variables. Their joint distribution is the cloud of 8939 points in three dimensions.

So what: A test example is predicted correctly exactly when $F_0 > F_1$. So the fraction of points with $F_0 > F_1$ is the test accuracy.

Epoch	mean F_0	mean F_1	mean F_2	fraction with $F_0 > F_1$
5000	94.625	114.731	95.851	0.005
12900	81.584	75.620	30.883	0.849
20000	143.343	126.546	72.213	1.000

Table 3.2: Seed 0, all 8939 test examples, three checkpoints.

These means match the saved `correct_mean`, `runner_up_mean` and `rest_mean_mean` histories exactly.

Reproduction check. The seed-0 test split was rebuilt as the code does it (`torch.manual_seed(0)`, then `get_dataloaders`). It gave 8939 examples. The population O-information at the three checkpoints came out as 0.4130, 0.4044 and 0.3471. These are exactly the values stored in `higher_mi_signal.json`. So the split, the triplets and the estimator are all reproduced.

3.8 The Code That Does This

The function is `compute_higher_mi_logit_triplet_for_checkpoint(model, test_loader, device)` in `06_Code/src/predictors/higher_mi.py`.

Step 1: Run the frozen checkpoint on every test batch.

Step 2: Keep only position 2: `logits = model.forward(x)[:, 2, :]`.

Step 3: Drop the "=" class: `logits[:, :p]`, with $p = 113$.

Step 4: F_0 = logit at the true label.

Step 5: Set the true-label logit to $-\infty$, take the maximum: this is F_1.

Step 6: $F_2 = (\text{sum} - F_0 - F_1)/(p-2)$.

Step 7: Concatenate over batches. The test set comes in 3 batches of at most 3830.

Why Step 3 matters. The "=" class was never a target. Its logit was never trained to be low or high in any controlled way. If it were kept, it could become a false runner-up, or it could shift the rest-mean. The code's own self-test puts a value of 99 in that slot and checks that it is ignored.

Chapter 4: O-Information (Brief)

This chapter is short on purpose. Information theory is already known to you. We only fix the formula, the sign convention and how the code uses it.

4.1 Definition

Definition 4.1 (O-information for three variables).

Why: We want one number that says how F_0, F_1, F_2 share information as a group. Pairwise mutual information cannot say this.

What: For three variables, O-information equals the interaction information. It compares what F_0 and F_1 share, with and without knowledge of F_2.

Formula: (Eq. 13 of the paper)

$$\Omega(F_0, F_1, F_2) = I(F_0; F_1) - I(F_0; F_1 \mid F_2) \tag{A}$$

So what:

$\Omega > 0$: redundancy. Knowing F_2 removes part of what F_0 and F_1 share. So the three carry overlapping information.

$\Omega < 0$: synergy. Knowing F_2 increases what F_0 and F_1 share. So some information exists only in the three together.

4.2 Term Extraction on Real Numbers

Relation (A) has two terms. Let us read both terms off the real seed-0 data, for one ordering of the variables.

Term (i): $I(F_0; F_1)$**.** This is the information shared by the correct logit and the runner-up logit, across the 8939 test examples.

Term (ii): $I(F_0; F_1 \mid F_2)$**.** Given that the rest-mean F_2 is known, the information still shared by F_0 and F_1 will be this term.

Epoch	Term (i) $I(F_0; F_1)$	Term (ii) $I(F_0; F_1 \mid F_2)$	(i) − (ii)
5000	0.4241	0.0006	0.4235
12900	0.6815	0.2754	0.4061
20000	2.8554	2.5001	0.3553

Table 4.1: Seed 0, one ordering $(F_0, F_1 \mid F_2)$, KSG with $k = 4$, in nats.

Substituting into (A) at epoch 12900:

$$\begin{aligned} \Omega &= I(F_0; F_1) - I(F_0; F_1 \mid F_2) \\ &= 0.6815 - 0.2754 \\ &= 0.4061 \end{aligned}$$

So what: Notice what happens at epoch 20000. Both terms are large, because F_0 and F_1 have become almost perfectly correlated (Pearson $r = 0.998$). But their difference is still a modest 0.36. So a large shared information does not by itself mean a large O-information. O-information measures only the part that F_2 explains away.

4.3 Averaging Over the Six Orderings

Why: In theory Ω is symmetric. Any of the three variables can play the role of "F_2". But the KSG *estimate* at a finite sample size is not exactly symmetric.

What: The code computes (A) for all $3! = 6$ orderings of (F_0, F_1, F_2). It reports the mean as the signal and the standard deviation as a diagnostic. This follows the caption of Fig. 7 of the paper.

Formula:

$$\Omega_{\text{pop}}(t) = \frac{1}{6} \sum_{\pi \in S_3} \Big[I(F_{\pi_1}; F_{\pi_2}) - I(F_{\pi_1}; F_{\pi_2} \mid F_{\pi_3}) \Big]_{\text{checkpoint } t}$$

So what: At epoch 12900 the single ordering gave 0.4061. The six-ordering mean is 0.4044, with standard deviation 0.0099. So the asymmetry of the estimator is small here. The function is `o_information_three_way(f0, f1, f2, k)`.

4.4 The Hypothesis Being Tested

The paper's story is about training dynamics.

At random initialisation, the three scores are unstructured, so synergy dominates and $\Omega < 0$.

When the model learns the rule, the scores become coordinated, so redundancy dominates and $\Omega > 0$.

The redundancy peak sits at the grokking transition.

For this thesis, the hypothesis to test is stronger:

The peak of $\Omega_{\text{pop}}(t)$ comes before the grok epoch, with a timing that follows the grok epoch across seeds.

Chapter 5: The KSG Estimator

5.1 Why an Estimator Is Needed

Why: F_0, F_1, F_2 are continuous real numbers. To compute $I(F_0;F_1)$ directly we would need their probability densities. We do not have them. We have only 8939 sample points.

One simple option is to cut each axis into bins and count. But the bin width is an arbitrary choice. The answer changes with it.

What: The Kraskov–Stögbauer–Grassberger (KSG) estimator avoids bins. It uses distances to nearest neighbours instead. The paper's Appendix A uses this estimator, and the code follows it exactly.

So what: No bin width has to be chosen. The only parameter is k, the number of neighbours.

5.2 How It Works, Step by Step

Let us take $I(X;Y)$ with T sample points (x_i, y_i).

Step 1: Distance. All distances use the Chebyshev (maximum) norm:

$$\big\|(x,y)-(x',y')\big\|_\infty = \max\big(|x-x'|,\,|y-y'|\big).$$

Step 2: The radius ε_i. For each point i, find its k-th nearest *other* point in the joint (x,y) space. Call that distance ε_i.

Step 3: Marginal counts. Given that the radius is ε_i, the number of other points strictly inside it along the x-axis alone will be:

$$N_x(i) = \#\big\{\, j \neq i :\ |x_j - x_i| < \varepsilon_i \,\big\}.$$

$N_y(i)$ is defined the same way on the y-axis.

Step 4: Combine with the digamma function ψ.

$$I(X;Y) = \psi(k) + \psi(T) - \big\langle \psi(N_x+1) + \psi(N_y+1) \big\rangle \tag{A7}$$

Here $\langle\cdot\rangle$ is the average over all T points.

For the conditional term the joint space is (x,y,z). The counts are taken in the z, (x,z) and (y,z) subspaces:

$$I(X;Y \mid Z) = \psi(k) + \big\langle \psi(N_z+1) - \psi(N_{xz}+1) - \psi(N_{yz}+1) \big\rangle \tag{A9}$$

So what: The idea in one line: if X and Y are strongly dependent, a small joint neighbourhood still contains few marginal neighbours, so the counts are small and the estimate is large.

5.3 Choice of k

The code uses $k = 4$ (`K_DEFAULT = 4`).

The paper states $k = 4$ only for transfer entropy.

It is reused here for O-information, for consistency.

This is an **assumption**, confirmed with Jonathan on 2 October 2026.

5.4 Code Functions

`_kth_nn_distances(points, k)`: finds ε_i with a `scipy cKDTree`, `p=np.inf`.
`_strict_count(points, eps)`: counts neighbours strictly inside ε_i.
`mutual_information_ksg(x, y, k)`: Eq. (A7).
`conditional_mutual_information_ksg(x, y, z, k)`: Eq. (A9).

5.5 Built-In Sanity Checks

Running `python higher_mi.py` directly runs a self-test on synthetic data. It checks known answers.

Two independent Gaussians: $I \approx 0$ (must be below 0.1).
$y = x+$ small noise: $I > 1$.
F_1 and F_2 both near-copies of F_0: $\Omega > 0$ (redundant).
$F_2 \approx F_0 \cdot F_1$ with independent F_0, F_1: $\Omega < 0$ (synergistic).
A tiny fake model with hand-picked logits: F_0, F_1, F_2 match hand calculation, and the "=" class is ignored.

These checks confirm that the sign convention and the triplet extraction are correct before the real run.

Chapter 6: Population O-Information versus Temporal O-Information

This chapter explains the biggest adaptation in the predictor. It is important for the thesis write-up and for the viva.

6.1 What "Samples" Means in the Paper

What: In the paper, one training sweep t gives one joint observation: the three class scores at that sweep. The KSG samples are the training sweeps. In Eq. (14) of the paper, T is the total number of training steps.

So what: So the paper's O-information is about *dynamics*. It asks: do the three output processes move together redundantly or synergistically as training goes on?

6.2 What "Samples" Means in This Project

Definition 6.1 (Population O-information, $\Omega_{\text{pop}}(t)$).

Why: An earlier draft (2 October 2026) used windows of training checkpoints as the samples. It had only about 100 samples per window. The window size was also an arbitrary choice. The KSG estimate would be noisy.

What: At one fixed checkpoint t, each test example gives one triplet (F_0, F_1, F_2). The KSG samples are the 8939 test examples. This gives one O-information value per checkpoint.

Formula:

$$\Omega_{\text{pop}}(t) = \Omega\big(F_0, F_1, F_2\big) \text{ estimated across the test population at checkpoint } t.$$

So what: The sample size is large (8939, not 100) and there is no window parameter. But it measures a different thing. It measures *population structure at one frozen moment*, not co-evolution in time.

Naming. The code docstring recommends always calling it "Population O-information". This keeps it clearly separate from the paper's (temporal) O-information.

6.3 The Risk: Shared Example Difficulty

Let us see why this change can matter.

All three numbers F_0, F_1, F_2 come from the same forward pass on the same input. So across the test population they may share a common cause: how "easy" or "hard" the example is.

An easy example: high F_0, lower F_1, very low F_2.
A confusing example: F_0 and F_1 close together.

A shared cause like this produces redundancy by itself. It can do so whether or not the model has grokked. Then $\Omega_{\text{pop}}(t)$ could stay positive throughout, with only its size changing.

The real data shows strong correlation among the three variables at every checkpoint.

Epoch	$r(F_0, F_1)$	$r(F_0, F_2)$	$r(F_1, F_2)$
5000	0.744	0.772	0.953
12900	0.858	0.756	0.798
20000	0.998	0.548	0.542

Table 6.1: Pearson correlations across the 8939 test examples, seed 0.

6.4 The Temporal Control

Definition 6.2 (Temporal O-information, Ω_{temp}).

Why: We do not want to settle the risk above only by argument. We want to test it on the data. So a second signal is computed that follows the paper's own meaning of "samples".

What: At each checkpoint, take the mean of F_0, F_1, F_2 over the test set. This gives three time series, each of length 401. Slide a window of $W = 100$ consecutive checkpoints along training time, with stride 10. Inside each window, the 100 checkpoints are the KSG samples.

Formula:

$$\begin{aligned}\text{number of windows} &= \left\lfloor \frac{401 - 100}{10} \right\rfloor + 1 \\ &= 30 + 1 \\ &= 31\end{aligned}$$

Each window value is placed at its centre epoch. The centres run from epoch 5000 to epoch 35000.

So what: This is an **exploratory control**. It is not a second candidate for the scored signal. Its only job is to be compared side by side with Ω_{pop}.

If the two curves tell the same story, the shared-difficulty confound is probably not dominating.
If they tell different stories, the two are not interchangeable, and the thesis should say so plainly.

Two properties of the control must be kept in mind:

One window covers 100 checkpoints, that is about 9900 epochs. So the control has a coarse time resolution.

The first window centre is epoch 5000. No value exists before that.

Both properties matter when we read the control's results in Chapter 11.

6.5 Transfer Entropy (Exploratory Only)

What: The paper also uses transfer entropy. The code computes it once per seed, in both directions, between the mean correct logit and the mean runner-up logit, for delays $\tau = 1, \ldots, 10$:

$$TE_{X \to Y}(\tau) = I\big(Y(t+\tau); X(t) \;\big|\; Y(t)\big).$$

It is estimated with the same KSG conditional-MI function.

So what: It gives no event epoch. It is not plotted with the other signals. It is not scored. It is stored only under `exploratory.transfer_entropy` in the signal file.

Chapter 7: The Event Rule: From a Curve to One Epoch

7.1 Why an Event Rule Is Needed

Why: The evaluation protocol compares one predictor epoch with one grok epoch per seed. But $\Omega_{\text{pop}}(t)$ is a whole curve of 401 values. So we need a fixed rule that picks one epoch from the curve.

What: The rule must be fixed *before* looking at its score. Otherwise the rule could be tuned to pass.

7.2 The Primary Rule: Global Maximum

Definition 7.1 (Primary event epoch P).

What: P is the checkpoint epoch at which $\Omega_{\text{pop}}(t)$ is largest.

Formula:

$$P = \arg\max_t \ \Omega_{\text{pop}}(t).$$

So what: This is the peak of redundancy. It is the paper's own landmark ("redundancy peak at grokking").

Why the maximum was chosen as primary.

Reason 1: Consistency. Every other primary rule in this project is a global extremum. AGE uses the NC1 minimum. HTSR uses the alpha minimum. Weight-PCA uses the effective-rank minimum. All were frozen on 17 September 2026.

Reason 2: Always defined. A global maximum always exists. The scoring needs one epoch for every seed. A rule that can return "none" breaks the scoring for that seed.

The rule was frozen on 3 October 2026. The function is `find_global_maximum_epoch(checkpoints, signal)`.

7.3 Exploratory Rules

Zero-crossing. The first checkpoint where Ω_{pop} goes from negative to non-negative, that is, synergy to redundancy. This matches the paper's description of its Fig. 7. But it returns `None` if the curve never crosses. It can also fire on noise. Function: `find_zero_crossing_epoch`.

Temporal-control maximum. The same global-maximum rule, applied to Ω_{temp}.

Exploratory rules are reported for the record. They are **not** part of the PASS/FAIL decision.

Chapter 8: The Frozen Evaluation Protocol

This protocol is the same one used for every predictor. It is restated here so that this document stands on its own.

8.1 Quantities

For seed s, let G_s be the grok epoch and P_s the primary event epoch. Let the margin be $m = 100$ epochs.

Definition 8.1 (Lead).

Why: A warning is useful only if it comes clearly before the event.

What: Seed s counts as a lead if the event comes more than 100 epochs before grok.

Formula:

$$\text{lead}_s = \mathbf{1}\big[P_s < G_s - 100\big], \qquad \text{lead_frac} = \frac{1}{5}\sum_s \text{lead}_s.$$

So what: We need lead_frac ≥ 0.8, that is, at least 4 of 5 seeds.

Definition 8.2 (Relative lead).

Why: A lead of 500 epochs means much more for a seed that groks at 6921 than for one that groks at 23747.

What: The lead is measured as a fraction of the grok epoch.

Formula:

$$\text{rel_lead}_s = \frac{G_s - P_s}{G_s}, \qquad \text{mean_rel_lead} = \frac{1}{5}\sum_s \text{rel_lead}_s.$$

So what: We need $0.05 \leq$ mean_rel_lead ≤ 0.9. Below 0.05, the warning is too late to be useful. Above 0.9, it fires almost at initialisation and carries no timing information.

Definition 8.3 (Rank correlation ρ).

Why: A real predictor should fire later on seeds that grok later.

What: ρ is the Spearman rank correlation between (G_s) and (P_s) over the 5 seeds. If all P_s are equal, ρ is set to 0.

So what: We need $\rho \geq 0.9$.

8.2 The PASS Rule

$$\text{PASS} \iff \text{lead_frac} \geq 0.8 \ \text{ and } \ \rho \geq 0.9 \ \text{ and } \ 0.05 \leq \text{mean_rel_lead} \leq 0.9.$$

8.3 Why $\rho \geq 0.9$: Calibration

The threshold was not chosen by hand. On 29 September 2026, 10000 random predictors $P_s \sim \text{Uniform}[0, 40000]$ were drawn. The 99th percentile of their ρ values was taken. It came out as 0.8999999999999998, written 0.9.

The rule itself was tested on five synthetic controls with known answers:

Control	Construction	Pass rate
A: ideal	$P = 0.5\,G$	100%
B: constant early	$P = 5000$	0%
C: post-grok	$P = 1.5\,G$	0%
D: noisy ideal, 25% noise	$P = 0.5\,G +$ noise	91.04%
E1: random	$P \sim U[0, 40000]$	0.22%
E2: random, always leads	$P \sim U[0, G]$	21.70%

Table 8.1: Synthetic-control validation of the rule (Run 2, ρ threshold at the 99th percentile).

Known caveat. With only 5 seeds, Spearman ρ can take only the values $-1, -0.8, \ldots, 0.8, 1$. Control E2 passes 21.70% of the time, because its sampling range grows with G. This caveat is recorded, not hidden. It does not affect Higher-MI, because Higher-MI fails on lead, not on ρ alone.

Chapter 9: The Whole Pipeline in One Picture

Every box below has now been defined. So the picture can be read from top to bottom without any new word.

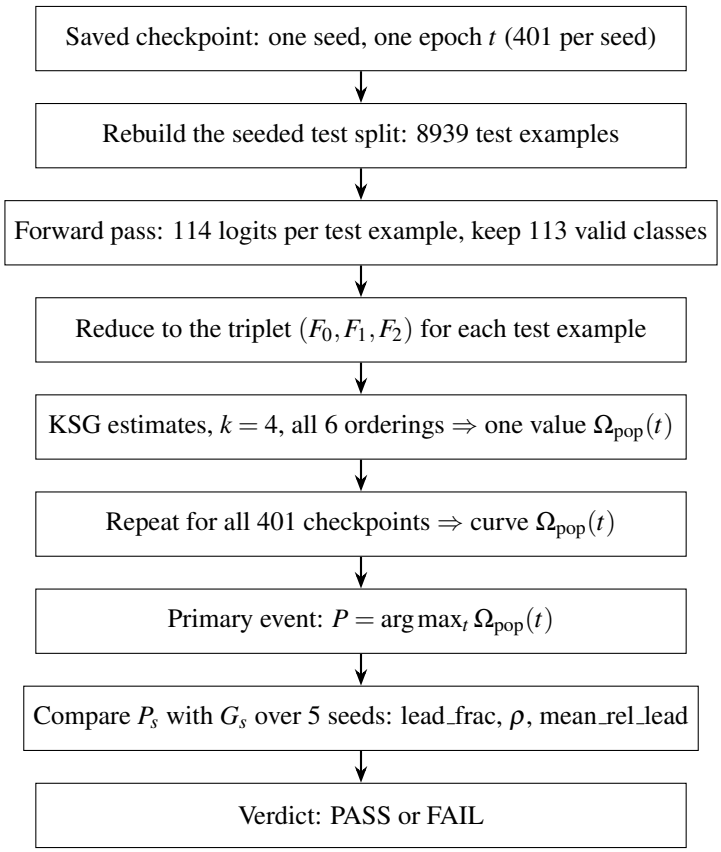


Figure 9.1: Higher-MI pipeline, checkpoint to verdict.

9.1 Where Each Box Lives in the Code

`06_Code/src/predictors/higher_mi.py` (new file)

`compute_higher_mi_logit_triplet_for_checkpoint`: boxes 3–4.

`mutual_information_ksg`, `conditional_mutual_information_ksg`, `o_information_three_way`: box 5.

`compute_population_o_information_signal`: box 6 (a batch version).

`find_global_maximum_epoch`, `find_zero_crossing_epoch`: box 7.

`compute_temporal_o_information_signal`: temporal control.

`transfer_entropy_ksg`, `transfer_entropy_table`: exploratory.

`06_Code/scripts/run_nanda_benchmark.py`

`_checkpoint_predictor_higher_mi(...)`: boxes 1–7 for one seed, writes the files.

Registered in `CHECKPOINT_PREDICTOR_FUNCS`; summary key `higher_mi_predictor`.

`06_Code/src/unified_measurements.py`

`higher_mi_dir`, `save_higher_mi_data(...)`: saves the arrays.

`06_Code/scripts/analyze_nanda_unified.py`

`build_events()`: reads P_s.

`PRIMARY_SIGNALS`: contains "Higher-MI: population omega maximum (implemented)".

`evaluate_frozen_protocol()`: boxes 8–9.

Writes `predictor_events.json` and `aggregate_higher_mi.json`.

`06_Code/scripts/plot_higher_mi_curves.py` (new file)

Raw-curve plots only, no event epochs, no PASS/FAIL.

Shared model versus predictor code. The shared model file `transformer_four_head.py` did not change. The training loop did not change. Higher-MI is computed only from checkpoints that already existed. No retraining was done.

9.2 Output Files per Seed

Each `seed_<N>/higher_mi/` folder has 10 files:

`higher_mi_checkpoints.npy`, `higher_mi_population_omega.npy`, `higher_mi_population_omega_std.npy`,
`higher_mi_correct_mean.npy`, `higher_mi_runner_up_mean.npy`, `higher_mi_rest_mean_mean.npy`,
`higher_mi_temporal_checkpoints.npy`, `higher_mi_temporal_omega.npy`, `higher_mi_temporal_omega_std.npy`,
`higher_mi_signal.json` (primary event at top level; exploratory items in a separate block).

Chapter 10: Two Bugs Found and Fixed

Both bugs were found while building the predictor. Both would have given wrong numbers without any crash. That is why they are recorded in detail.

10.1 Bug 1: Inclusive versus Strict Neighbour Counting

Setup. The KSG counts $N_x(i)$ must use a *strict* inequality: $|x_j - x_i| < \varepsilon_i$. The paper says "strictly less than" in Section II.D.

What went wrong. `scipy`'s `cKDTree.query_ball_point(r=eps)` counts points with distance $\leq \varepsilon_i$. That is inclusive.

Why it matters here. ε_i is a maximum-norm distance to the k-th joint neighbour. For that neighbour, one coordinate is the "binding" one. Along that coordinate's own axis, the neighbour sits at exactly ε_i. So an inclusive count wrongly includes it. This was not rare. It happened for about half the points in a cross-check run.

Size of the error. In one test case, the inclusive version changed Ω by about 2.6 times. On $N = 500$ synthetic points, 248 of 500 counts differed from the correct strict counts.

Fix. Query with the largest floating-point number strictly below ε_i:

$$r = \texttt{np.nextafter}(\varepsilon_i, 0).$$

Then "$\leq r$" is exactly the same as "$< \varepsilon_i$".

Verification. The fixed tree version matched a brute-force $O(N^2)$ strict-count reference *bit for bit* on $N = 500$ points.

Side benefit: speed. A dense distance-matrix version took about 19 seconds per checkpoint. For 5×401 checkpoints that is about 10.7 hours. The tree version takes about 0.7 seconds per checkpoint. The full real run took about 27 minutes.

10.2 Bug 2: The Test Split Was Not Re-Seeded

Setup. The train/test split is made by `random_split`. It uses PyTorch's global random-number generator. During training, `train_one_seed()` calls `torch.manual_seed(seed)` before `get_dataloaders()`. So each seed has its own fixed split.

What went wrong. The recompute path, `recompute_from_checkpoints()`, builds a `test_loader` *without* seeding first. So that loader holds a different random split. Many of its "test" examples were actually training examples for that seed.

Why it matters here. For the other predictors this did not matter. HTSR and Weight-PCA ignore `test_loader`. Spectral and AGE rebuild their own seeded train loader. But Higher-MI is entirely about the test population. Before grokking, training examples have far higher confidence than test examples. Mixing them in would bias F_0, F_1, F_2. The bias is worst on the early checkpoints, which are exactly the ones a leading indicator needs. There would be no error message.

Fix. `_checkpoint_predictor_higher_mi` ignores the passed-in loader. It reads the seed from `seed.npy`, calls `torch.manual_seed(seed)` and `np.random.seed(seed)`, and then rebuilds the loader. This follows the same precedent as the Spectral and AGE wrappers.

Verification. The rebuilt seed-0 split has 8939 examples. It reproduces the stored Ω_{pop} values exactly (Section 3.7).

Chapter 11: Results

11.1 Raw Curves First

Before any scoring, the raw curves were plotted and inspected. This was a deliberate project practice: “report raw $\Omega(t)$ before trusting event epochs”.

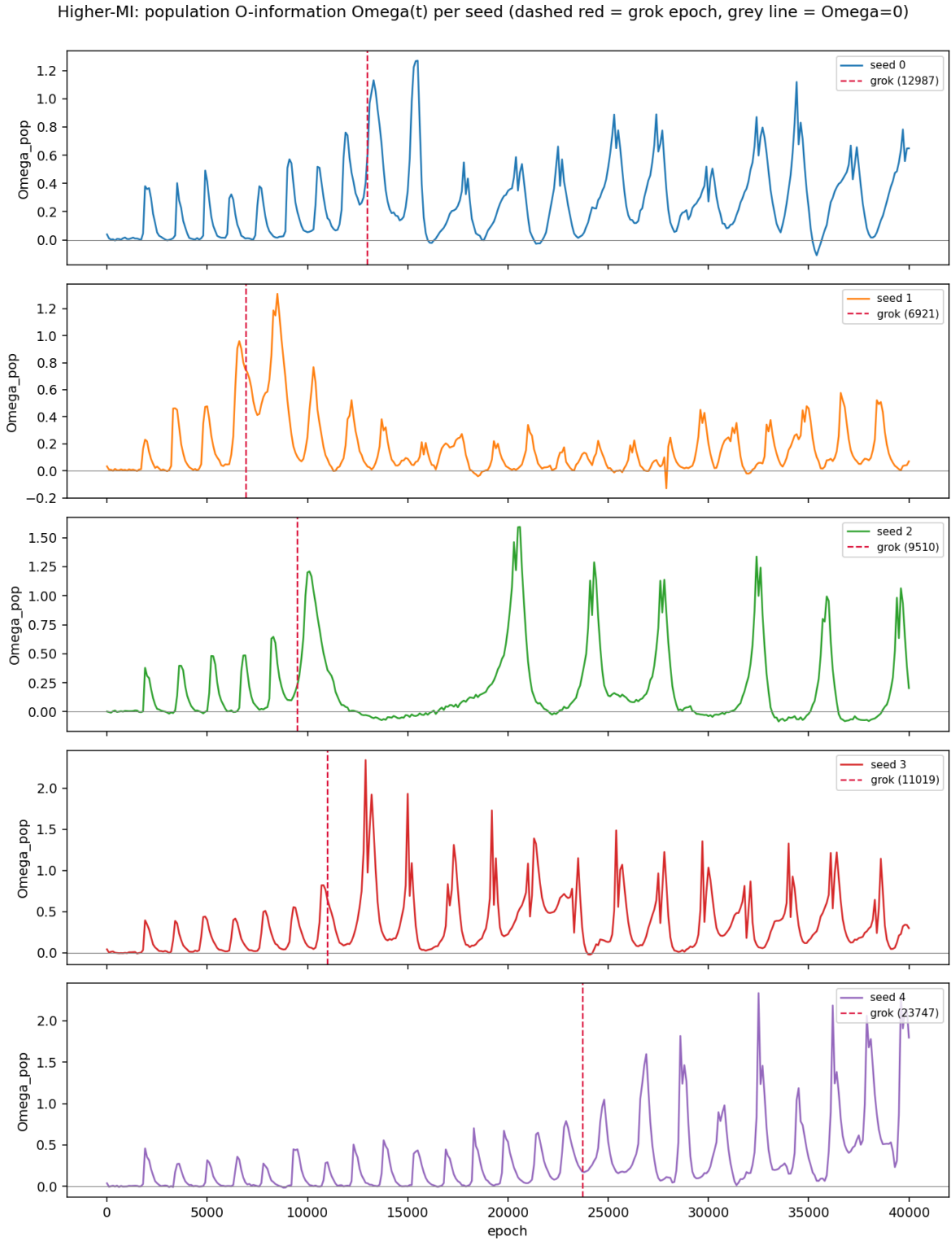


Figure 11.1: $\Omega_{\text{pop}}(t)$ for all five seeds. Dashed red line: grok epoch. Grey line: $\Omega = 0$. File: `reports/higher_mi_population_omega_all_seeds.png`.

What do we see in Figure 11.1?

Observation 1: No strongly negative start. At epoch 0, Ω_{pop} is between $+0.002$ and $+0.040$ in all seeds. The paper's "strongly negative at initialisation" regime is not seen. Negative values are rare and small, the lowest being -0.130.

Observation 2: Repeated spikes for the whole run. The curve is not one rise and one peak. It spikes up and falls back towards 0 again and again. This happens before grok and after grok.

Observation 3: The first spike is at the same epoch in every seed. In all five seeds, Ω_{pop} first exceeds 0.2 at exactly epoch 1900. After that, the pre-grok spikes come roughly every 1400 to 1600 epochs. This timing does not depend on the seed's grok epoch, which ranges from 6921 to 23747.

Observation 4: No visible landmark at grok. The grok line falls in the middle of the oscillating pattern. It does not mark a clear change.

Observation 5: Amplitude drifts, differently per seed. Seed 4 rises from 0.037 to 1.797, about 48 times. Seed 1 goes only from 0.033 to 0.070.

Observation 3 is a strong hint. A clock that is identical in all seeds cannot be tracking grokking, because grokking happens at very different times in different seeds. The cause of this common clock was not investigated. It is recorded as an observation only.

Seed	G	Ω_{pop} at epoch 0	last value	minimum	max value	checkpoints with $\Omega_{\text{pop}} < 0$
0	12987	0.0388	0.6484	−0.1092	1.2689	20 of 401
1	6921	0.0332	0.0704	−0.1296	1.3089	17 of 401
2	9510	0.0024	0.2037	−0.0834	1.5917	115 of 401
3	11019	0.0399	0.2978	−0.0238	2.3403	13 of 401
4	23747	0.0374	1.7972	−0.0163	2.3347	20 of 401

Table 11.1: Summary of the raw population curves.

11.2 Population versus Temporal Control

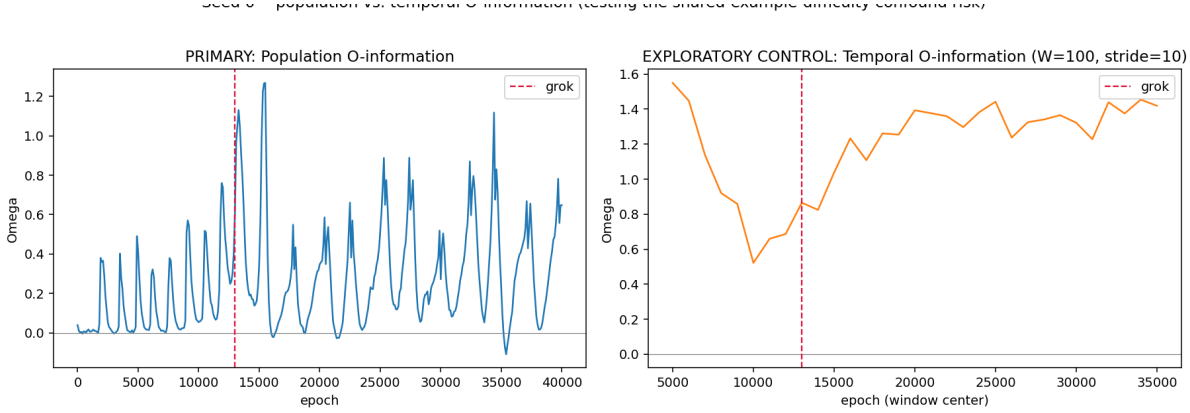


Figure 11.2: Seed 0. Left: $\Omega_{\text{pop}}(t)$ (primary). Right: Ω_{temp} (exploratory control, $W = 100$, stride 10). File: `reports/higher_mi_population_vs_temporal_seed0.png`.

Left panel. The repeated-spike pattern described above.

Right panel. Much smoother. It starts high, about 1.55 at epoch 5000. It falls to a minimum of about 0.52 near epoch 10000. That is a little before the grok epoch 12987. Then it rises and stays between about 1.2 and 1.45. It never becomes negative in any seed.

So what: The two curves have clearly different shapes. So they are not two copies of one "shared difficulty" factor. This partly answers the confound concern of Chapter 6. But it also shows that popula-

tion and temporal O-information are *not* interchangeable. The temporal control shows a soft dip before grok. The population signal, which is the scored one, shows no such feature.

11.3 Primary Event per Seed

Seed	G_s	P_s	$\Omega_{\text{pop}}(P_s)$	$P_s - G_s$	P_s/G_s
0	12987	15500	1.2689	2513	1.19
1	6921	8500	1.3089	1579	1.23
2	9510	20600	1.5917	11090	2.17
3	11019	12900	2.3403	1881	1.17
4	23747	32500	2.3347	8753	1.37

Table 11.2: Primary event (global maximum of Ω_{pop}). The peak is after grok in every seed.

11.4 Frozen-Protocol Scoring

The verdict depends on three quantities. Let us write the main relation and then take out each term.

$$\text{PASS} \iff \underbrace{\text{lead_frac} \geq 0.8}_{\text{(i)}} \wedge \underbrace{\rho \geq 0.9}_{\text{(ii)}} \wedge \underbrace{0.05 \leq \text{mean_rel_lead} \leq 0.9}_{\text{(iii)}} \tag{A}$$

Term (i): lead_frac

Why: We check whether the peak comes more than 100 epochs before grok.

What: Seed by seed, the test $P_s < G_s - 100$:

Seed 0: $15500 < 12887$? No.
Seed 1: $8500 < 6821$? No.
Seed 2: $20600 < 9410$? No.
Seed 3: $12900 < 10919$? No.
Seed 4: $32500 < 23647$? No.

Formula:

$$\text{lead_frac} = \frac{0}{5} = 0.$$

So what: Not a single seed gives an early warning.

Term (ii): ρ

Why: We check whether later-grokking seeds have later peaks.

What: Rank the five G_s and the five P_s separately (1 = smallest).

Seed	0	1	2	3	4
rank of G_s	4	1	2	3	5
rank of P_s	3	1	4	2	5
difference d_s	1	0	−2	1	0
d_s^2	1	0	4	1	0

Formula:

$$\begin{aligned}\rho &= 1 - \frac{6\sum_s d_s^2}{n(n^2-1)} \\ &= 1 - \frac{6 \times 6}{5 \times 24} \\ &= 1 - \frac{36}{120} \\ &= 0.7\end{aligned}$$

So what: The peak timing does follow the grok timing partly. Seed 1 is earliest in both and seed 4 is latest in both. But seeds 2 and 3 swap places. $0.7 < 0.9$, so this term also fails. The Pearson correlation is 0.8976, but the protocol uses rank correlation.

Term (iii): mean_rel_lead

Why: We check how early, as a fraction of each seed's grok epoch, the peak comes.

What: For each seed, $\text{rel_lead}_s = (G_s - P_s)/G_s$:

$$\text{Seed 0: } \frac{12987 - 15500}{12987} = \frac{-2513}{12987} = -0.1935$$
$$\text{Seed 1: } \frac{6921 - 8500}{6921} = \frac{-1579}{6921} = -0.2281$$
$$\text{Seed 2: } \frac{9510 - 20600}{9510} = \frac{-11090}{9510} = -1.1661$$
$$\text{Seed 3: } \frac{11019 - 12900}{11019} = \frac{-1881}{11019} = -0.1707$$
$$\text{Seed 4: } \frac{23747 - 32500}{23747} = \frac{-8753}{23747} = -0.3686$$

Formula:

$$\begin{aligned}\text{mean_rel_lead} &= \frac{1}{5}\Big[(-0.1935) + (-0.2281) + (-1.1661) + (-0.1707) + (-0.3686)\Big] \\ &= \frac{-2.1270}{5} \\ &= -0.4254\end{aligned}$$

So what: Negative means the event is after grok. On average, the peak lands about 43% of a grok-time *after* grok.

Final Substitution into (A)

$\text{lead_frac} = 0$	≥ 0.8 ?	No
$\rho = 0.7$	≥ 0.9 ?	No
$\text{mean_rel_lead} = -0.4254$	$\in [0.05, 0.9]$?	No

$$\boxed{\text{VERDICT} = \text{FAIL}}$$

These are exactly the values stored in `aggregate_higher_mi.json` under `frozen_protocol_verdict`.

11.5 Exploratory Signals (Not Scored)

These are given for completeness. They do not change the verdict.

Signal	P_s for seeds 0–4	lead_frac	ρ	mean_rel_lead
Population zero-crossing	500, 1600, 300, 1100, 200	1.0	-0.7	0.9181
Temporal-control maximum	5000, 5000, 19000, 5000, 14000	0.8	see below	0.1703

Table 11.3: Exploratory event rules, scored here with the same three quantities only for comparison.

Zero-crossing. It fires very early in every seed, between epoch 200 and 1600. This is only the curve wobbling around 0 near initialisation. Its ρ is negative. Its mean_rel_lead is 0.918, which is above 0.9. This is the same failure pattern as synthetic Control B (constant early signal).

Temporal-control maximum. It passes lead_frac ($4/5$) and mean_rel_lead (0.170). It fails on ρ. Three of the five seeds have their maximum at epoch 5000. Epoch 5000 is the *first* window centre (Section 6.4). So in those seeds the maximum is at the left edge of the curve. That "lead" comes from where the curve starts, not from a grok-related event. Because of the three tied values, the rank correlation is weak. The tie-aware Spearman value is 0.11. The value 0.00 stored in `aggregate_higher_mi.json` comes from `rank_corr()`, which breaks ties by position. Either way, it is far below 0.9.

Transfer entropy (exploratory). At delay $\tau = 1$, the estimate from the mean correct logit to the mean runner-up logit is larger than the reverse in all five seeds.

Seed	$TE_{F_0 \to F_1}(\tau{=}1)$	$TE_{F_1 \to F_0}(\tau{=}1)$	$TE_{F_0 \to F_1}(\tau{=}10)$	$TE_{F_1 \to F_0}(\tau{=}10)$
0	0.4463	0.2752	0.2175	0.2152
1	0.6125	0.3267	0.1654	0.1737
2	0.3299	0.3026	0.3088	0.3300
3	0.4713	0.3812	0.2050	0.3495
4	0.4285	0.3394	0.3077	0.3480

Table 11.4: Transfer entropy on the 401-point mean-logit trajectories (nats).

This is computed on whole trajectories that change strongly during training. So it should not be read as a clean causal result. It is kept only as a record.

Chapter 12: What the Negative Result Means

12.1 The Main Conclusion

Population O-information on the logit triplet does not lead grokking in this benchmark. Its peak comes after grok in all 5 seeds. It is the 7th predictor to close negative, after L2 Norm, Dropout, Spectral, AGE, HTSR Alpha and Weight-PCA.

12.2 What the Raw Curves Tell Us

Not a grokking-specific signal. The spikes start at epoch 1900 in every seed. They repeat through the whole run, before and after grok. Their size grows in a way that does not follow the grok epoch. So Ω_{pop} seems to follow general training progress, such as how the logit margins develop. It does not mark the grokking transition.

The paper's sign story is not reproduced. The paper sees strongly negative O-information at initialisation and a switch to positive at grokking. Here Ω_{pop} starts near 0 and is positive almost all the time.

12.3 Comparison with the Paper's Own Claim

Paper: the redundancy peak *coincides* with grokking (MPS model, Fashion-MNIST, 3 classes).

This benchmark: the peak is not even coincident. It comes 1.17 to 2.17 times the grok epoch.

So: the paper's claim was never a "leads" claim. Our stricter test shows that the signal does not become a leading indicator when it is transferred to the standard grokking transformer.

12.4 What the Result Does Not Mean

It does not mean the paper is wrong in its own setting. The model, the data and the meaning of "samples" are all different there.

It does not mean O-information is useless for studying learning. It means that this adapted form, under this protocol, gives no early warning.

It does not mean the pipeline is broken. The same protocol passes the ideal synthetic predictor 100% of the time (Chapter 8). The estimator passes its own known-answer tests (Chapter 5). The stored values are reproduced exactly from the checkpoints (Section 3.7).

12.5 Thesis Wording (Ready to Use)

Higher-MI (Pomarico et al., 2025) was adapted to modular addition as a population O-information over the correct-class, runner-up and rest-mean logits of the 8939 held-out examples at each checkpoint. Under the frozen unified protocol, the primary event (global maximum of the signal) occurred after the grok epoch in all five seeds (lead_frac $= 0$*,* $\rho = 0.7$*, mean relative lead* $= -0.43$*). The raw signal oscillates throughout training with a seed-independent onset at epoch 1900, which suggests that it tracks general training dynamics rather than the grokking transition. Higher-MI is therefore closed as negative.*

Chapter 13: Assumptions, Limitations and Open Items

13.1 Assumptions Made in the Adaptation

A1. Three-variable reduction. 113 logits are reduced to (F_0, F_1, F_2). Another reduction could give a different signal.

A2. Population reading. Samples are test examples at one checkpoint, not training steps. This changes what is measured (Chapter 6).

A3. $k = 4$. Stated in the paper only for transfer entropy.

A4. Dropping the "=" class. The never-trained output is removed before the reduction.

A5. Global maximum as the primary rule. Chosen for consistency with the other predictors and because it is always defined.

13.2 Limitations

Five seeds. Spearman ρ takes only 11 possible values with $n = 5$. The Control E2 caveat applies to the protocol in general (Chapter 8).

Checkpoint spacing. Checkpoints are 100 epochs apart. A feature narrower than 100 epochs cannot be seen.

Temporal control resolution. One window spans about 9900 epochs, and the first value is at epoch 5000.

Shared-difficulty confound. It is partly addressed by the temporal control. It is not fully removed.

Cause of the epoch-1900 onset. Not investigated.

13.3 Open Items

Run log location. The full success log of the 5-seed run exists only in the cloud container used for the heavy compute. It is not at `results/nanda_unified/higher_mi_run.log`. The instruction was to save it there only if a seed failed, and none did. So this is optional, not a defect.

Commit state. The Higher-MI work of 3 October 2026 was left uncommitted on instruction. This question is still open.

Next predictor. Commutator Defect (Predictor 8) is next. It is not started.

Chapter 14: How to Reproduce

All commands are run from the repository root. They use only saved checkpoints. No retraining happens.

Step 1: Recompute the signal for 5 seeds.

```
python 06_Code/scripts/run_nanda_benchmark.py
--predictors higher_mi --seeds 5 --overwrite higher_mi
--output_dir 08_Experiments/results/nanda_unified
```

Step 2: Plot the raw curves.

```
python 06_Code/scripts/plot_higher_mi_curves.py
```

Step 3: Score against the frozen protocol.

```
python 06_Code/scripts/analyze_nanda_unified.py
```

Step 4 (optional): Run the estimator self-test.

```
python 06_Code/src/predictors/higher_mi.py
```

Two command-line points.

`--seeds` takes a *count*, not a list. So `--seeds 5` means seeds 0 to 4. The form `--seeds 0 1 2 3 4` is not valid.

`analyze_nanda_unified.py` takes no arguments. It always scores every known signal from `08_Experiments/results/nanda_unified`.

Do not re-run `run_control_experiment.py` for this. The threshold $\rho \geq 0.9$ is already frozen.

Expected console line per seed:

```
[seed N] higher_mi recomputed from 401 of 401 saved checkpoints
(eval_every=100, no retrain)
```

Appendix A: Symbols

Symbol	Meaning
(a,b)	input pair, $a,b \in \{0,\ldots,112\}$
c	correct class, $c = (a+b) \bmod 113$
$\ell[j]$	logit of class j for one test example
F_0	correct-class logit, $\ell[c]$
F_1	runner-up logit, $\max_{j\neq c} \ell[j]$
r	runner-up class
F_2	mean logit of the remaining 111 classes
t	checkpoint epoch
Ω	O-information, $I(F_0;F_1) - I(F_0;F_1 \mid F_2)$
$\Omega_{\text{pop}}(t)$	population O-information at checkpoint t (primary signal)
Ω_{temp}	temporal O-information on windows of mean logits (exploratory control)
k	number of nearest neighbours in KSG, here 4
ε_i	max-norm distance from point i to its k-th joint neighbour
$N_x(i)$	number of other points strictly within ε_i along x
ψ	digamma function
T	number of KSG samples (8939 for Ω_{pop}; 100 for Ω_{temp})
G_s	grok epoch of seed s
P_s	primary event epoch of seed s, $\arg\max_t \Omega_{\text{pop}}(t)$
m	lead margin, 100 epochs
ρ	Spearman rank correlation of (G_s) and (P_s)